



MODUL 10 · VALIDASI DATA & FORMS

Zod Schema & Inference Advanced



omit, pick, partial, extend, discriminatedUnion — DRY schema patterns untuk project nyata.

Schema Derivation — DRY Principle

Satu base schema, banyak turunan. Perubahan model = update satu tempat saja.

● ● ● omit, pick, partial, extend

```
const postSchema = z.object({
  id:      z.string().cuid(),
  title:   z.string().min(3).max(100),
  slug:    z.string().regex(/^[a-z0-9-]+$ /),
  content: z.string().min(10),
  published: z.boolean(),
  viewCount: z.number(),
  createdAt: z.date(),
  authorId: z.string().uuid(),
})

// — Untuk CREATE: hapus auto-generated fields —————
const createPostSchema = postSchema.omit({
  id:true, viewCount:true, createdAt:true,
})

// — Untuk UPDATE: semua optional kecuali id —————
const updatePostSchema = postSchema
  .partial() // semua jadi optional
  .required({ id:true }) // kecuali id tetap wajib

// — Untuk LIST preview: hanya field yang diperlukan —————
const postPreviewSchema = postSchema.pick({
  id:true, title:true, slug:true, published:true, createdAt:true,
})

// — Extend: tambah field ke schema yang ada —————
const postWithAuthorSchema = postSchema.extend({
  author: z.object({ name:z.string(), avatar:z.string().nullable() }),
})

// Types — tidak perlu tulis manual!
type CreatePostInput  = z.infer<typeof createPostSchema>
type UpdatePostInput  = z.infer<typeof updatePostSchema>
type PostPreview      = z.infer<typeof postPreviewSchema>
```

discriminatedUnion & z.enum

Type-safe branching berdasarkan discriminator field. Ideal untuk response API.

● ● ● z.enum – nilai terbatas

```
// z.enum – konstanta string
const roleSchema = z.enum(["ADMIN", "AUTHOR", "READER"])
type Role = z.infer<typeof roleSchema>
// "ADMIN" | "AUTHOR" | "READER"

// Dari TypeScript enum:
enum Status { DRAFT="draft", PUBLISHED="published" }
const statusSchema = z.nativeEnum(Status)

// z.literal – nilai persis
const activeSchema = z.literal(true)
const adminRole = z.literal("ADMIN")

// z.union – salah satu dari beberapa
const idSchema = z.union([
  z.string().uuid(),
  z.number().int().positive(),
])
```

● ● ● discriminatedUnion – API response

```
const apiResponseSchema = z.discriminatedUnion(
  "status",
  [
    z.object({
      status: z.literal("success"),
      data: z.object({
        id: z.string(),
        title: z.string(),
      }),
    }),
    z.object({
      status: z.literal("error"),
      code: z.number(),
      message: z.string(),
    }),
  ]
)
type ApiResponse =
  z.infer<typeof apiResponseSchema>

// Usage:
const res = apiResponseSchema.parse(data)
if (res.status === "success") {
  console.log(res.data.id) // ✓ typed!
} else {
  console.log(res.message) // ✓ typed!
}
```

superRefine — Multiple Errors Sekaligus

Untuk validasi kompleks yang perlu menambahkan error ke beberapa field berbeda.

```
● ● ● superRefine — password validation kompleks

const registerSchema = z.object({
  email: z.string().email("Email tidak valid"),
  password: z.string().min(8, "Min 8 karakter"),
  confirmPassword: z.string(),
}).superRefine(({ password, confirmPassword }, ctx) => {
  // Bisa addIssue ke beberapa field sekaligus!
  if (confirmPassword !== password) {
    ctx.addIssue({
      code: z.ZodIssueCode.custom,
      message: "Password tidak cocok",
      path: ["confirmPassword"], // error di field confirmPassword
    })
  }
  if (!/[A-Z]/.test(password)) {
    ctx.addIssue({
      code: z.ZodIssueCode.custom,
      message: "Harus mengandung huruf kapital",
      path: ["password"], // error di field password
    })
  }
  if (!/[0-9]/.test(password)) {
    ctx.addIssue({
      code: z.ZodIssueCode.custom,
      message: "Harus mengandung angka",
      path: ["password"],
    })
  }
})
// .flatten().fieldErrors:
// { password: ["Harus mengandung huruf kapital", "Harus mengandung angka"],
//   confirmPassword: ["Password tidak cocok"] }
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Apa yang dilakukan `postSchema.omit({ id: true, createdAt: true })`?

- A) Hapus field id dan createdAt dari database B) Buat schema baru tanpa field id dan createdAt — untuk form CREATE C) Jadikan field optional

Q2 — Apa keunggulan `z.discriminatedUnion` dibanding `z.union` untuk API response?

- A) Lebih cepat B) TypeScript bisa narrow type berdasarkan discriminator — type-safe branching di if/switch C) Support lebih banyak types

Q3 — `superRefine` dipakai untuk?

- A) Validasi yang lebih ketat dari refine B) Menambahkan multiple errors ke beberapa field berbeda sekaligus C) Menjalankan validasi async

Tugas Mandiri

Bangun schema library yang reusable untuk seluruh project.

01 `lib/validations/post.ts`

Buat postSchema lengkap. Derive: createPostSchema (omit), updatePostSchema (partial+required id), postPreviewSchema (pick). Export semua types.

02 `lib/validations/user.ts`

Buat registerSchema dengan superRefine: password===confirmPassword, ada huruf kapital. Buat loginSchema: email + password.

03 `discriminatedUnion`

Buat ActionResult<T> discriminatedUnion: {success:true, data:T} | {success:false, errors:Record<string,string[]>}. Gunakan sebagai return type Server Actions.

04 `Indonesian errors`

Update semua schema dengan pesan error bahasa Indonesia yang jelas. Test: parse invalid → error messages tepat dan dapat dipahami user.



Ekspor semua schema DAN types dari validation files. One source of truth untuk seluruh project.

Yang Sudah Kita Pelajari

- ✓ omit, pick, partial, extend: derive schema baru dari base — DRY, satu perubahan update semua.
- ✓ z.enum untuk nilai terbatas. z.discriminatedUnion untuk type-safe API response branching.
- ✓ refine(): satu error per field. superRefine(): multiple errors ke beberapa fields.
- ✓ z.coerce.number(): konversi string URL params ke number sebelum validasi.
- ✓ Satu base schema → banyak turunan. Types ikut otomatis. Tidak ada duplikasi.

→ Selanjutnya: **Bab 3 — Validasi di Server Actions & API Routes**

